

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

AI Smart Food Supply Orchestrator (ASF-Orchestrator) 소프트웨어 요구사항 명세서

수요·공급·유통 동적 최적화를 위한 AI 가동형 통합 제어 플랫폼

문서 번호 : ASF-REQS-SRS-001

버전 : 1.0.0 (공식 배포본)

작성일자 : 2026년 6월 5일

작성자 : Generative AI Tech Lead / Product Owner

승인자 : 농림축산식품 IT 부서 인프라 아키텍트

목차 (Table of Contents)

1. 개요 (Introduction)

- 1.1 목적 및 범위
- 1.2 용어 정의 및 약어
- 1.3 시스템 참조 구성도

2. 시스템 아키텍처 및 시스템 전체 설명

- 2.1 제품의 조망 (Product Perspective)
- 2.2 하드웨어 및 인프라 인터페이스 인터랙션
- 2.3 사용자 세그먼트 및 주요 제약사항

3. 기능 요구사항 (Functional Requirements)

- 3.1 [FR-1] 실시간 데이터 파이프라인 및 피처 저장소
- 3.2 [FR-2] TFT 기반 가격 및 수급 예측 엔진 서빙
- 3.3 [FR-3] 다목적 최적화 초개인화 추천 시스템
- 3.4 [FR-4] VRP 기반 친환경 공동 물류 및 경로 최적화
- 3.5 [FR-5] Dynamic Pricing 및 ESG 그린 리워드 제어 시스템
- 3.6 [FR-6] API 게이트웨이 및 코어 인증

4. 외부 인터페이스 요구사항 (External Interface Requirements)

- 4.1 사용자 인터페이스 (User Interfaces)
- 4.2 소프트웨어 및 시스템 인터페이스

5. 비기능 요구사항 (Non-Functional Requirements)

- 5.1 성능 및 지연시간 요구사항 (Performance & Latency)
- 5.2 보안 및 프라이버시 요구사항 (Security & Privacy)
- 5.3 고가용성 및 복구 요구사항 (Availability & Fault Tolerance)
- 5.4 관측 가능성 및 모니터링 (Observability)

6. 데이터베이스 구조 및 데이터 모델 명세 (Data Architecture)

- 6.1 물리 데이터베이스 스키마 설계 (PostgreSQL DDL)
- 6.2 Redis 실시간 메모리 키 구조 설계

7. 주요 컴포넌트 인터페이스 API 명세 (API Contracts)

- 7.1 [B2C 모바일] AI 추천 장바구니 조회 및 갱신 API
- 7.2 [B2B 물류] 배송 경로 최적화 API

8. 부록: 수학적 정밀 수식 및 알고리즘 의사코드

1. 개요 (Introduction)

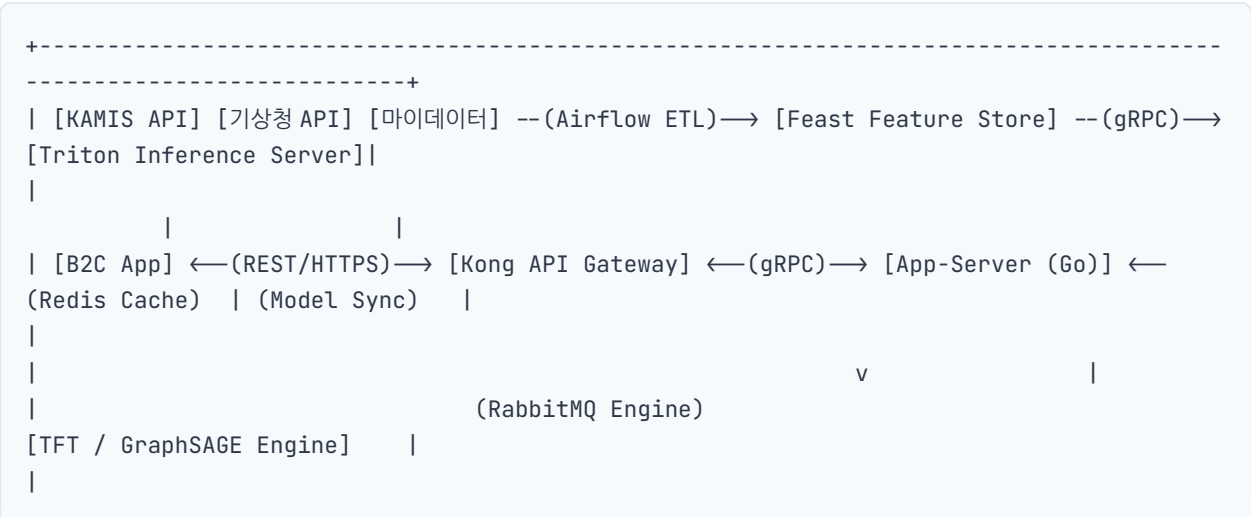
1.1 목적 및 범위

본 문서는 농림축산식품 공모전 출품 및 시스템 구현을 위한 'AI Smart Food Supply Orchestrator (ASF-Orchestrator)' 플랫폼의 소프트웨어 요구사항 명세서(SRS)입니다. 본 시스템은 기존에 수동적이었던 장보기 프로세스에서 벗어나, 시장의 수요(초개인화 식단 추천)와 공급망 데이터(도소매 유통가, 기상 및 산지 폐기 위험군)를 실시간으로 결합·조율(Orchestrate)하는 다목적 AI 통합 제어망 설계를 핵심 범위로 합니다. 주요 모듈은 실시간 데이터 수집 ETL 파이프라인, Feature Store 인하우스 구성, 시계열 TFT 추론 엔진, Multi-Objective 추천기, VRP 솔버, 그리고 REST/gRPC 통신 인프라를 포괄합니다.

1.2 용어 정의 및 약어

용어 / 약어	정의 및 상세 설명
ASF-Orchestrator	AI Smart Food Supply Orchestrator의 약어로, 본 플랫폼 시스템의 공식 명칭입니다.
TFT	Temporal Fusion Transformer. 시계열 예측에 특화된 어텐션 기반 딥러닝 아키텍처입니다.
Feast	실시간 스트리밍 및 배치 학습을 위한 프로덕션급 오픈소스 Feature Store 시스템입니다.
VRP	Vehicle Routing Problem. 차량의 적재 및 이동 거리를 최소화하는 고도의 조합 최적화 경로 탐색 기법입니다.
PostGIS	PostgreSQL 관계형 DB에 위치 지리적 공간 인덱스(GIST) 및 지하 공간 정보 쿼리를 지원하는 오픈소스 플러그인입니다.

1.3 시스템 참조 구성도



```
v
|
|          [VRP Solver Core (C++)] <---> [PostgreSQL / PostGIS (Spatial
DB)]
|
+-----+
-----+
```

2. 시스템 아키텍처 및 시스템 전체 설명

2.1 제품의 조망 (Product Perspective)

본 시스템은 마이크로서비스 아키텍처(MSA) 및 도메인 주도 설계(DDD) 패턴에 기반하여 구성됩니다. 모바일 단말에서 인입되는 B2C 트래픽 및 B2B 유통망 대시보드는 비동기 AMQP 큐(RabbitMQ)와 고성능 Go Core 비즈니스 API 레이어, 메모리 내 캐싱 저장소(Redis Cluster)를 통해 유기적으로 병목 현상 없이 처리됩니다.

2.2 하드웨어 및 인프라 인터페이스 인터랙션

- **GPU Compute Node:** Triton Inference Server가 가동되는 NVIDIA T4/A10G 환경에서 추론 가속화를 적용합니다.
- **GIS Database Node:** PostgreSQL 16 물리 스토리지에 PostGIS 확장을 올려 지리적 위경도 데이터를 위상 구조로 처리합니다.
- **In-Memory Layer:** Redis Enterprise 클러스터를 가동하여 실시간 동적 분산 가격 락(Lock)을 5ms 이하 응답속도로 핸들링합니다.

2.3 사용자 세그먼트 및 주요 제약사항

- **B2C 스마트 소비자:** 즉각적인 초개인화 장바구니 추천을 받아야 하며 단말기 API 대기 시간은 150ms를 준수해야 합니다.
- **B2B 유통 및 지자체 관리자:** 산지 폐기율, 수급 이상 알림, 최적 배차 정보를 대형 웹 UI에서 실시간으로 관제해야 합니다.
- **제약사항:** 마이데이터(개인 식생활, 건강, 병원 진료 연동 이력)는 국내 개인정보보호 가이드라인을 통과할 수 있도록 필드 레벨에서 복호화 불가능한 일방향 해시 및 양방향 AES-256-GCM 알고리즘으로 분리 암호화해야 합니다.

3. 기능 요구사항 (Functional Requirements)

3.1 [FR-1] 실시간 데이터 파이프라인 및 피처 저장소 (Data Ingestion & Feature Store)

- [FR-1.1 수집 자동화]: Apache Airflow 대문을 활용해 매일 새벽 04:00에 농산물유통정보(KAMIS), 산지 공판장 수급 내역, 기상청 예측 정보 API를 스크래핑하는 배치를 실행합니다.
- [FR-1.2 Feast 동기화]: 추출 데이터셋을 실시간으로 Feast Online Redis에 적재하여 딥러닝 추론 피처 조회 지연(Latency)을 10ms 이하로 유지합니다.

3.2 [FR-2] TFT 기반 가격 및 수급 예측 엔진 서빙 (Forecasting Engine Serving)

- [FR-2.1 단기 시계열 예측]: Temporal Fusion Transformer(TFT)를 통해 주요 15개 과잉/수급 민감 품목의 14일 뒤 단기 가격과 공급량의 임계 이탈 여부(Oversupply Risk Index, 0.0 ~ 1.0)를 추출합니다.
- [FR-2.2 Triton 연동]: Triton Inference Server 상의 ONNX 런타임에 모델 파일(.onnx)을 탑재하여 병렬 동시 요청 추론을 30ms 이내에 처리합니다.

3.3 [FR-3] 다목적 최적화 초개인화 추천 시스템 (Multi-Objective Optimization Engine)

- [FR-3.1 수학적 목적 함수 수립]: 추천 후보군은 사용자의 건강 정보 지수 및 로컬 농가와의 거리, 실시간 산지 폐기 위험 인덱스를 종합 고려해 연산됩니다.
- [FR-3.2 식단 템플릿 제어]: GraphSAGE 임베딩에 의해 1차 분류된 농산물 목록 중, 목적 함수 연산 결과 상위 K개 제품군을 조리 레시피 룰셋과 병합하여 JSON 스트림으로 전달합니다.

3.4 [FR-4] VRP 기반 친환경 공동 물류 및 경로 최적화 (Logistics Optimization Engine)

- [FR-4.1 배송 위경도 클러스터링]: B2C 주문 마감 시간 직후, 배송 위경도 데이터를 PostGIS를 통해 권역별 클러스터로 바인딩합니다.
- [FR-4.2 C++ 솔버 최적 경로]: 클러스터화된 배송 목적지들을 C++ 하이퍼 최적화 VRP 알고리즘에 전달하여 전체 운행 거리와 탄소 발생량을 최소화하는 최적 차량 배차 경로를 1분 이내에 계산합니다.

3.5 [FR-5] Dynamic Pricing 및 ESG 그린 리워드 제어 시스템 (Dynamic Incentives Engine)

- [FR-5.1 실시간 요율 연산]: 산지 공급 과잉률이 상승할 경우 해당 제품의 최종 판매가를 실시간으로 감면하는 실시간 할인 요율 적용 로직을 실행합니다.
- [FR-5.2 리워드 마일리지]: 탄소 감축량이 많은 로컬 푸드 위주의 농산물 구매 결제 시 탄소 중립 기여 포인트를 1원 단위로 지갑 스키마에 즉시 자동 정산 및 누적합니다.

3.6 [FR-6] API 게이트웨이 및 코어 인증 (API Gateway & Core Authentication)

- [FR-6.1 Gateway 라우팅]: 모바일 앱과 B2B Admin 웹의 통합 입구인 Kong API Gateway를 구성하여 JSON Web Token(JWT) 검증과 API 쓰로틀링(IP당 120회/분) 정책을 강제합니다.

4. 외부 인터페이스 요구사항 (External Interface Requirements)

4.1 사용자 인터페이스 (User Interfaces)

- **B2C 모바일 UI:** 다이내믹 AI 추천 장바구니 식단 렌더링 화면, ESG 그린 마일리지 누적 대시보드 화면이 반응형 네이티브 UI 구조로 구현됩니다.
- **B2B 웹 관제 UI:** 지자체 공무원 및 농가 사업자를 위한 지도 기반 공간 GIS 대시보드로, 배차 트럭 이동 실시간 모니터링을 지원합니다.

4.2 소프트웨어 및 시스템 인터페이스

- **정부 공공 API 커넥터:** 공공데이터포털 RESTful API와 OpenAPI 표준 포맷(XML/JSON) 통신 규격을 준수하여 연동합니다.
- **PG 결제 연동 모듈:** 토스페이먼츠 및 카카오페이 가맹 API 연동을 위한 웹훅 수신 핸들러 구조를 내장합니다.

5. 비기능 요구사항 (Non-Functional Requirements)

5.1 성능 및 지연시간 요구사항 (Performance & Latency)

- **지연을 보장:** 모든 REST API 엔드포인트의 응답시간은 p95 기준 150ms 이내여야 합니다.
- **동시성 부하 조절:** 최대 동시 접속자 10,000명 돌파 시 CPU 부하가 70%를 넘지 않도록 K8s Horizontal Pod Autoscaler(HPA)가 자동으로 트래픽 인스턴스를 무중단 스케일 아웃하도록 설계합니다.

5.2 보안 및 프라이버시 요구사항 (Security & Privacy)

- **암호화 알고리즘:** 수집되는 사용자 영양 및 진료 이력 데이터는 DB 블록 수준에서 AES-256-GCM 방식을 채택합니다.
- **보안 전송:** 모든 패킷 네트워크 전송 구간은 TLS 1.3 표준 암호화 터널을 의무화합니다.

5.3 고가용성 및 복구 요구사항 (Availability & Fault Tolerance)

- **서비스 가용 시간:** 연간 무중단 서비스율 99.9% 이상을 목표로 하며 가용 세그먼트 배포는 롤링 업데이트 규칙을 준수합니다.
- **장애 복구 대책 (DR):** PostgreSQL 주 DB 노드를 실시간 동기 슬레이브 노드로 리플리케이션 처리하고, 매일 오전 03:00에 콜드 클라우드 스토리지로 덤프 백업 스냅샷을 생성 전송합니다.

5.4 관측 가능성 및 모니터링 (Observability)

- **프로메테우스 통합:** 개별 마이크로서비스 엔진의 내부 가동 메모리, 스레드 풀, 큐 대기 길이를 `/metrics` REST API 포맷으로 내보냅니다.
- **분산 추적 시스템:** OpenTelemetry 표준 Trace ID 전파 기술을 통하여 마이크로서비스 간에 병목 지점을 시각 분석할 수 있도록 설계합니다.

6. 데이터베이스 구조 및 데이터 모델 명세 (Data Architecture)

6.1 물리 데이터베이스 스키마 설계 (PostgreSQL DDL)

```
-- 1. 사용자 정보 마이데이터 테이블
CREATE TABLE users (
    user_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email VARCHAR(100) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    household_size INT DEFAULT 1,
    health_target_keywords VARCHAR(255)[], -- 기저질환, 식단 조건 (예: 'diabetes', 'keto')
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 2. 농산물 마스터 및 수급 정보 테이블
CREATE TABLE agricultural_products (
    product_code VARCHAR(20) PRIMARY KEY,
    product_name VARCHAR(100) NOT NULL,
    category_name VARCHAR(50) NOT NULL,
    origin_location VARCHAR(100) NOT NULL,
    base_unit VARCHAR(10) NOT NULL,
    base_price DECIMAL(10, 2) NOT NULL,
    oversupply_risk_index DECIMAL(3, 2) DEFAULT 0.00, -- 0.00 ~ 1.00 수급 과잉도
    carbon_emissions_factor DECIMAL(5, 2) NOT NULL -- kg CO2 per kg
);

-- 3. 시계열 가격 예측 이력 테이블
CREATE TABLE forecast_pricing (
    forecast_id BIGSERIAL PRIMARY KEY,
    product_code VARCHAR(20) REFERENCES agricultural_products(product_code) ON DELETE CASCADE,
    target_date DATE NOT NULL,
    p10_predicted_price DECIMAL(10, 2) NOT NULL,
    p50_predicted_price DECIMAL(10, 2) NOT NULL,
    p90_predicted_price DECIMAL(10, 2) NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT uq_forecast UNIQUE (product_code, target_date)
);

-- 4. B2C 주문서 및 ESG 리워드 정산 연동 테이블
CREATE TABLE orders (
    order_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(user_id) ON DELETE RESTRICT,
    total_amount DECIMAL(12, 2) NOT NULL,
    discount_amount DECIMAL(12, 2) DEFAULT 0.00,
```

```

    earned_esg_points INT DEFAULT 0,
    delivery_coordinate GEOMETRY(Point, 4326) NOT NULL, -- 공간 지리 인덱스 (Spatial Index)
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 공간 쿼리 최적화를 위한 지리적 인덱스 생성
CREATE INDEX idx_orders_delivery_coordinate ON orders USING GIST(delivery_coordinate);

```

6.2 Redis 실시간 메모리 키 구조 설계

실시간으로 가격이 변동하는 동적 가격(Dynamic Pricing) 요율 및 사용자의 실시간 가용 장바구니 정보 구조 명세입니다.

```

# 1. 품목별 실시간 동적 할인 요율 캐시 (TTL: 1시간)
Key: dynamic_discount:product:<product_code>
Type: Hash
Fields:
  - current_price: "2450.00"
  - discount_rate: "0.15"
  - raw_oversupply_index: "0.78"
  - updated_at: "2026-06-05T12:00:00Z"

# 2. 사용자별 장바구니 AI 추천 결과 캐시 (TTL: 12시간)
Key: ai_basket:user:<user_id>
Type: String (JSON Encoded String)
Value:
{
  "basket_id": "f81d4fae-7dec-11d0-a765-00a0c91e6bf6",
  "calculated_at": "2026-06-05T06:15:30Z",
  "recommended_items": [
    {"product_code": "AGRI-CABB-001", "quantity": 2, "applied_discount": 0.20},
    {"product_code": "AGRI-POTA-012", "quantity": 1, "applied_discount": 0.10}
  ],
  "expected_esg_points": 350
}

```

7. 주요 컴포넌트 인터페이스 API 명세 (API Contracts)

7.1 [B2C 모바일] AI 추천 장바구니 조회 및 갱신 API (REST / HTTPS)

GET /api/v1/recommendation/basket

```
// HTTP 200 OK Response JSON
{
  "status": "success",
  "data": {
    "user_id": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",
    "basket_id": "c73a884d-2a1f-4cf6-ba64-50a7c936df31",
    "summary": {
      "total_items_count": 2,
      "estimated_original_price": 7500.00,
      "estimated_discounted_price": 5550.00,
      "total_discount_rate": 0.26,
      "estimated_esg_points": 350
    },
    "items": [
      {
        "product_code": "AGRI-CABB-001",
        "product_name": "유기농 괴산 양배추",
        "quantity": 1,
        "unit": "개",
        "original_price": 3000.00,
        "discounted_price": 2400.00,
        "oversupply_risk_level": "HIGH",
        "nutrition_match_reason": "사용자 식이섬유 섭취 최적화 결과 매칭",
        "esg_score_contribution": 150
      },
      {
        "product_code": "AGRI-ONIO-053",
        "product_name": "햇 양파",
        "quantity": 1,
        "unit": "망",
        "original_price": 4500.00,
        "discounted_price": 3150.00,
        "oversupply_risk_level": "VERY_HIGH",
        "nutrition_match_reason": "전체 조리 가이드라인 베이스 양념 필수 식단 구성 매칭",
        "esg_score_contribution": 200
      }
    ]
  }
}
```

```
}
```

7.2 [B2B 물류] 배송 경로 최적화 API (gRPC / Protocol Buffers)

```
syntax = "proto3";

package logistics.v1;

service LogisticsService {
  rpc CalculateRoute(CalculateRouteRequest) returns (CalculateRouteResponse);
}

message Coordinate {
  double latitude = 1;
  double longitude = 2;
}

message DeliveryDestination {
  string destination_id = 1;
  Coordinate coordinate = 2;
  double demand_weight_kg = 3;
}

message CalculateRouteRequest {
  string warehouse_id = 1;
  Coordinate warehouse_coordinate = 2;
  repeated DeliveryDestination destinations = 3;
  double vehicle_max_capacity_kg = 4;
}

message OptimizedRoute {
  int32 route_index = 1;
  repeated string path_destination_ids = 2;
  double total_distance_meters = 3;
  double total_travel_time_seconds = 4;
  double co2_emitted_kg = 5;
}

message CalculateRouteResponse {
  string warehouse_id = 1;
  repeated OptimizedRoute optimized_routes = 2;
  double base_co2_reduction_percentage = 3;
}
```

8. 부록: 수학적 정밀 수식 및 알고리즘 의사코드

소비자의 영양 선호 유틸리티, 공급 과잉률 보상, 유통 거리 페널티를 종합 결합하는 탐색 알고리즘입니다.

$$f(u, i) = \alpha \cdot \text{Utility}(u, i) + \beta \cdot \text{Oversupply}(i) - \gamma \cdot \text{FoodMile}(i)$$

```
def select_optimized_basket(user_id, candidate_items, k_limit, alpha, beta, gamma):
    # 다목적 최적화 함수를 기반으로 상위 K개의 장바구니 추천 품목을 그리디 방식으로 최종 선택합니다.
    scored_items = []

    for item in candidate_items:
        # 1. 유저 선호 점수 및 영양 균형 점수 산출
        utility = compute_user_nutrition_utility(user_id, item['product_code'])

        # 2. 실시간 공급 과잉 수준 지수 확보
        oversupply = item['oversupply_risk_index']

        # 3. 로컬푸드 산지로부터 배송지까지의 물류 탄소 발생 거리 가중치 계산
        food_mile = calculate_geographical_distance_km(user_id, item['origin_location'])

        # 4. 종합 스코어 도출 (Multi-Objective Optimization Score)
        total_score = (alpha * utility) + (beta * oversupply) - (gamma * food_mile)

        scored_items.append({
            "product_code": item['product_code'],
            "score": total_score,
            "utility": utility,
            "oversupply": oversupply,
            "food_mile": food_mile
        })

    # 종합 스코어가 가장 높은 순으로 정렬 후 상위 K개 결과 슬라이싱 반환
    scored_items.sort(key=lambda x: x['score'], reverse=True)
    return scored_items[:k_limit]
```